Encoding Categorical Data: Ordinal and Label Encoding

Contents

1. Overview . . . 2
2. Numerical and categorical data . . . 3
    2.1 Nominal data . . . 3
    2.2 Ordinal data . . . 3
3. Why categories must become numbers . . . 4
    3.1 Why ordinal encoding is only for ordered categories . . . 4
4. Ordinal encoding versus label encoding . . . 5
5. How ordinal encoding works . . . 6
6. Ordinal encoding in scikit-learn . . . 7
    6.1 The customer data . . . 7
    6.2 Which encoder for which column . . . 8
    6.3 Split before encoding . . . 9
    6.4 The categories parameter . . . 9
    6.5 Fit on the training set, transform both . . . 10
7. Label encoding in scikit-learn . . . 11
    7.1 Only for the target . . . 11
    7.2 Encoding the purchased column . . . 12
8. Summary . . . 14
9. Sources . . . 14
10. Key terms . . . 15

Where this fits: Pipeline map step 5 (Engineer features). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	**5 Features**	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Train-test split; Feature transformation.
- **Leads to:** One-hot encoding; Column transformer.
- **Compare with:** One-hot encoding; Binning and binarization.

1. Overview

Key point: Ordinal encoding turns the ordered categories of a feature into the numbers 0, 1, 2, ...; **label encoding** (G-1030) does the same for the categories of the target.

Recall the three basic terms. A **feature** (G-772) is an input variable, one column of the data table. The **target** (G-1949) is the output we predict. An **observation** (G-1374) is one record, one row of the table. Feature transformation changes a feature's form so a model can use it. **Standardization** (G-1874) covered one of its jobs, feature scaling. This Note covers another job, which comes up in almost every ML problem: **encoding categorical data**, that is, turning **categories** (G-354) into numbers.

Figure 1 shows the whole topic. The kind of categorical data decides the technique:

- nominal features get **one-hot encoding** (G-1379; one column per category);
- ordinal features get **ordinal encoding** (G-1404) (sections 5 and 6);
- a categorical target gets label encoding (section 7).

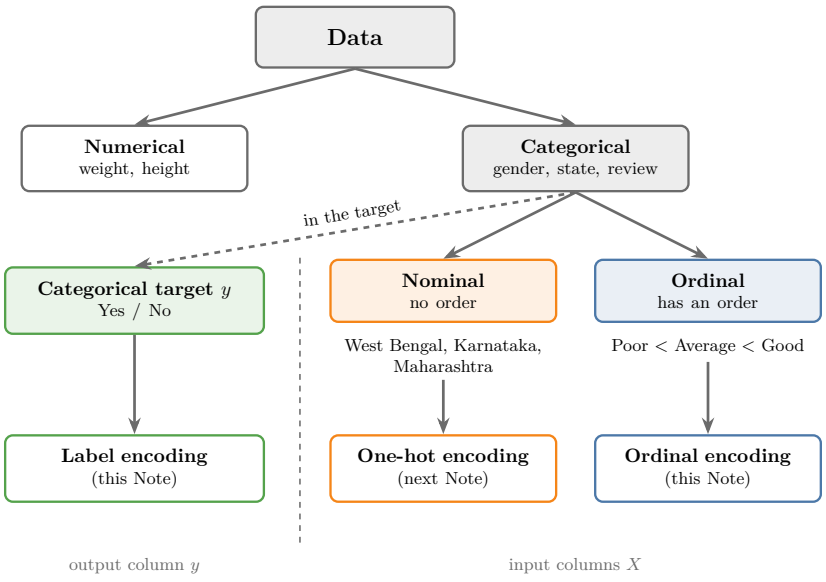


Figure 1: Types of data and the encoding technique each one needs

2. Numerical and categorical data

> **Key point:** Data is either numerical (numbers) or categorical (**categories** (G-354)); categorical data is either nominal (no order) or ordinal (has an order).

Every column is either **numerical data** (G-1367) or **categorical data** (G-351; see numerical and categorical data). Categorical data itself comes in two types, nominal and ordinal.

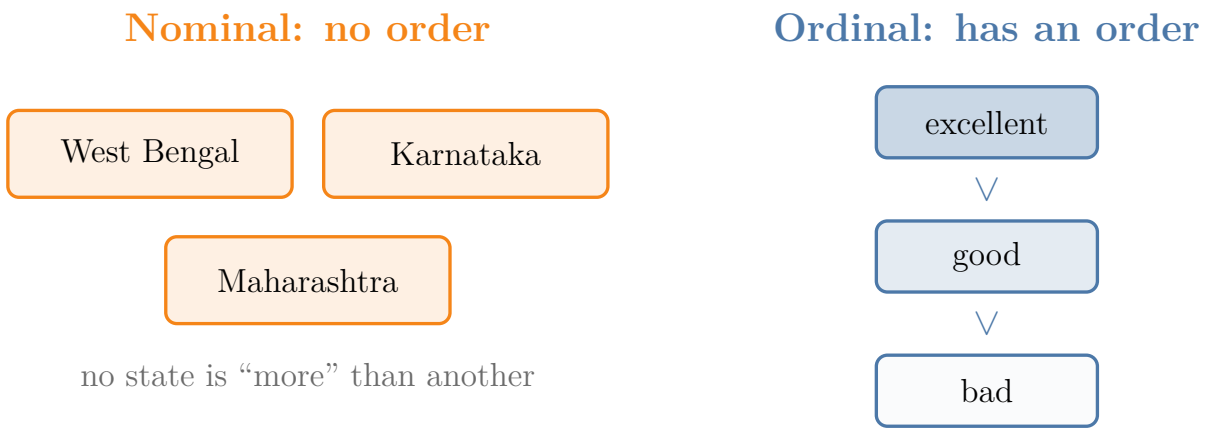


Figure 2: Nominal categories sit side by side; ordinal categories stack in order

Figure 2 contrasts the two types with the examples below.

2.1 Nominal data

> **Key point:** Nominal **categories** (G-354) have no relationship and no order among them.

In **nominal data** (G-1330), the categories have no relationship with each other and no order. A good example is a column of Indian states: West Bengal, Karnataka, Maharashtra. We cannot say that one state is more or greater than another.

Engineering branches are another example. Computer science, mechanical and civil are different branches, but none of them is "higher" than the others.

2.2 Ordinal data

> **Key point:** Ordinal **categories** (G-354) have a natural order, so we can rank them.

In **ordinal data** (G-1403), there is a relationship between the categories: they can be put in order. A product review is a good example: if the possible answers are excellent, good and bad, then excellent > good > bad.

Education level works the same way. Post-graduate (PG) is higher than under-graduate (UG), which is higher than high school.

3. Why categories must become numbers

> **Key point:** Categorical data is usually stored as text, but ML algorithms work only with numbers.

Categorical data is mostly stored as strings, such as `"Male"` or `"Good"`. ML algorithms expect numbers. So it is our job to convert the **categories** (G-354) into numbers before training.

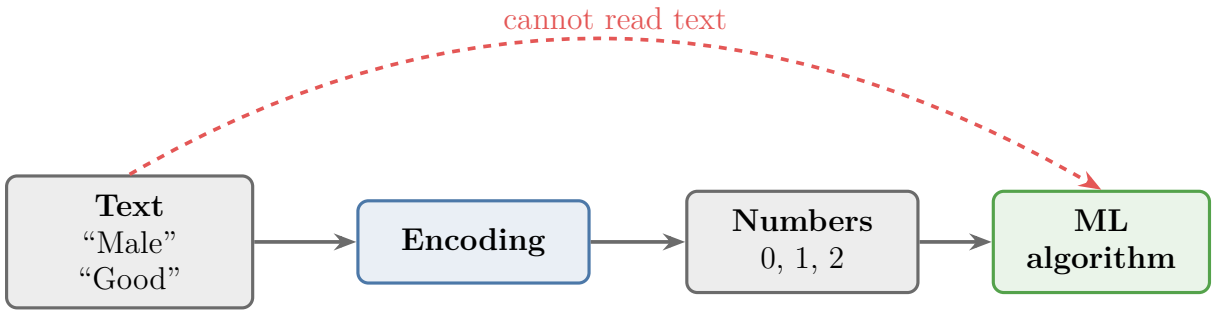


Figure 3: Encoding sits between text categories and the algorithm

Figure 3 shows where encoding sits.

There are many ways to do this, called **encoding** techniques. The two most popular ones are:

1. **Ordinal encoding** (G-1404), used on **ordinal data** (G-1403): the subject of this Note.
2. One-hot encoding, used on **nominal data** (G-1330): the subject of the next Note.

This Note also covers a third technique, **label encoding** (G-1030). Label encoding does the same job as ordinal encoding, but it is meant for the **target** (G-1949) instead of the features.

3.1 Why ordinal encoding is only for ordered categories

> **Key point:** Numbers carry an order. Put on nominal **categories** (G-354), they make a model group the categories by an order that does not exist.

Take the three states of section 2.1 and number them alphabetically: Karnataka 0, Maharashtra 1, West Bengal 2. The numbers now say that Maharashtra lies between the other two, which means nothing for states.

A model takes the numbers at face value. Figure 4 shows it for a **decision tree** (G-561; a model that asks yes/no questions such as "is the code below 1.5?", taught in a decision tree is nested if-else). Each question cuts the number line at one point:

1. A cut at 0.5 puts Karnataka on one side, and Maharashtra with West Bengal on the other.

2. A cut at 1.5 puts Karnataka with Maharashtra, and West Bengal alone.
3. No cut can put Karnataka with West Bengal and leave Maharashtra alone, because 1 lies between 0 and 2.

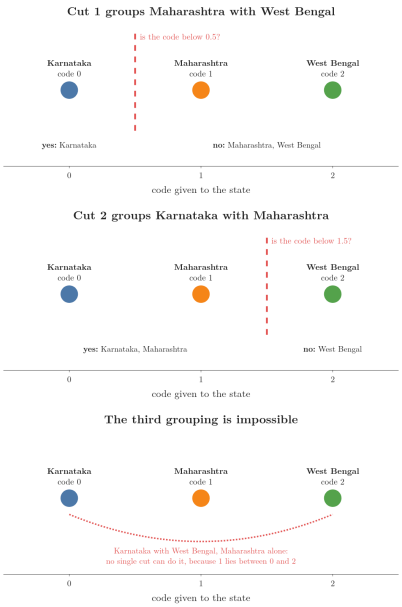


Figure 4: Three states numbered 0, 1, 2. One cut of the number line can form only two of the three possible groupings; which two depends only on the numbers we happened to choose. Idea after StatQuest, "One-Hot, Label, Target and K-Fold Target Encoding".

Which groupings are possible depends only on the numbers we happened to give. With **ordinal data** (G-1403) the same behaviour is what we want: School, UG, PG really do lie in that order, so "below UG" is a sensible group. So ordinal encoding is for ordinal features only; nominal features get one-hot encoding (see why nominal data needs its own technique).

4. Ordinal encoding versus label encoding

> **Key point:** Use **ordinal encoding** (G-1404) for ordinal features (X); use **label encoding** (G-1030) for a categorical target (y).

Any dataset for supervised learning has features and a **target** (G-1949). The features together are called X, and the target is called y. The two encoders differ in which of these they are for:

- **Features (X):** if a feature holds ordinal **categories** (G-354), we apply ordinal encoding.
- **Target (y):** if the target is categorical, we apply label encoding.

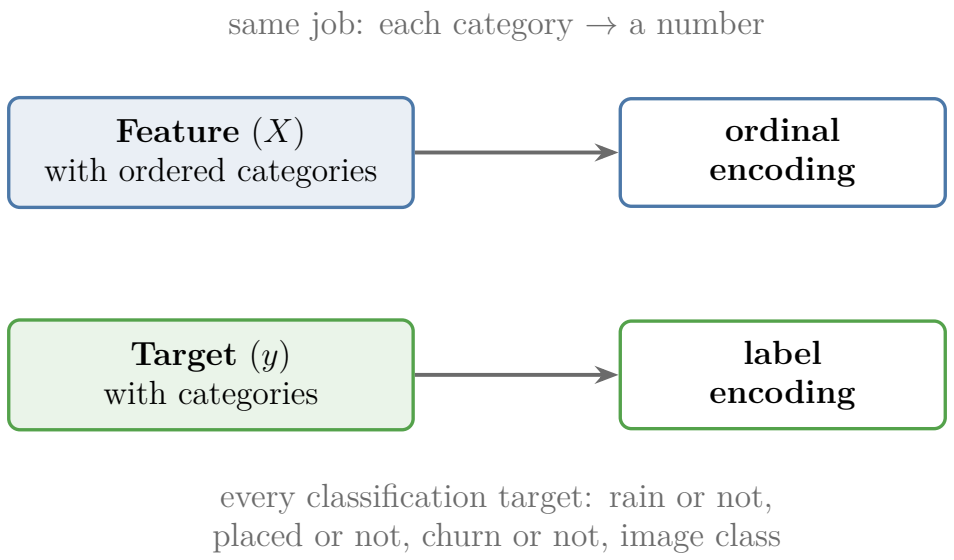


Figure 5: Ordinal encoding for ordered features, label encoding for a categorical target

Figure 5 shows the rule: the same job, applied to different columns.

A categorical target is what every **classification** (G-395; predicting a category) problem has. Examples: will it rain today or not; will a student get placed or not; will a customer leave (customer churn) or not; which class does an image belong to.

Label encoding does the same thing as ordinal encoding: it replaces each category with a number. Label encoding is simply designed for the target's labels, which is where its name comes from. The column it is meant for is the main difference between the two.

5. How ordinal encoding works

> **Key point:** We tell the encoder the order of the **categories** (G-354); it gives the lowest category 0, the next 1, and so on.

Suppose a column `education` holds three values, over many rows: high school, under-graduate (UG) and post-graduate (PG). We want to turn it into a numerical column.

First we decide whether the column is nominal or ordinal. If it were nominal, we would use one-hot encoding. Here PG > UG > high school, so the column is ordinal and we use **ordinal encoding** (G-1404).

Next we tell the encoder the order: high school is the lowest, UG is in the middle, and PG is the highest. The encoder then replaces each category with its position in that order:

Category	Code
high school	0

Category	Code
UG	1
PG	2

The top half of Figure 6 shows this on a few rows, with the label "School" for high school. Every row with School becomes 0, every UG becomes 1, every PG becomes 2.

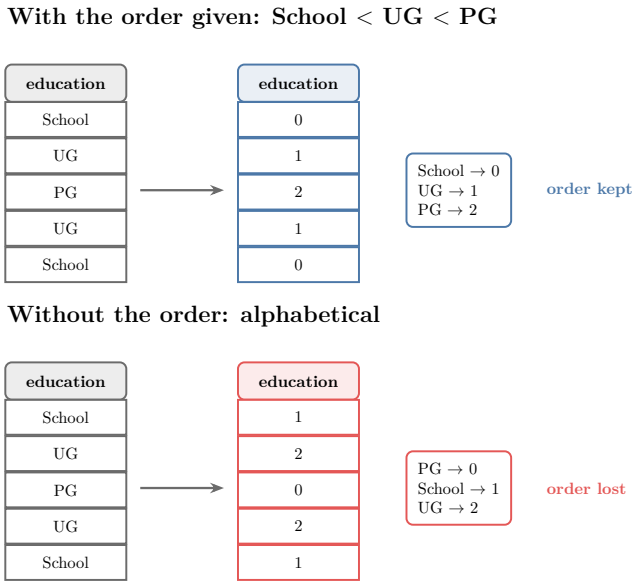


Figure 6: Ordinal encoding of the education column, with the order given (top) and without it (bottom)

The encoder cannot work out the order by itself. The encoder has no idea what "UG" or "PG" mean, so we must always tell it which category is lowest and which is highest.

6. Ordinal encoding in scikit-learn

> **Key point:** Split the data, then let `OrdinalEncoder` learn the **categories** (G-354) from the training set (in the order we give) and apply them to both sets.

6.1 The customer data

> **Key point:** 50 customers, with their age, gender, product review, education, and whether they bought a recommended product.

The data has 50 customers. When each customer bought a product, some other products were recommended to them; the `purchased` column says whether the customer then bought one of them (Yes or No).

age	gender	review	education	purchased
30	Female	Average	School	No
68	Female	Poor	UG	No
70	Female	Good	PG	No

`purchased` is the target, so the task is a classification problem. Each customer is one **observation** (G-1374). Every column except `age` is categorical.

6.2 Which encoder for which column

> **Key point:** Gender is nominal, review and education are ordinal, and the **target** (G-1949) purchased is categorical.

Before encoding, we identify the type of every categorical **feature** (G-772) and of the target (Figure 7):

- **gender** (Female, Male): there is no order between male and female, so it is nominal. Gender needs one-hot encoding.
- **review** (Poor, Average, Good): ordinal, so it needs **ordinal encoding** (G-1404).
- **education** (School, UG, PG): ordinal, so it needs ordinal encoding.
- **purchased** (Yes, No): nominal, and it is the target, so it needs **label encoding** (G-1030).

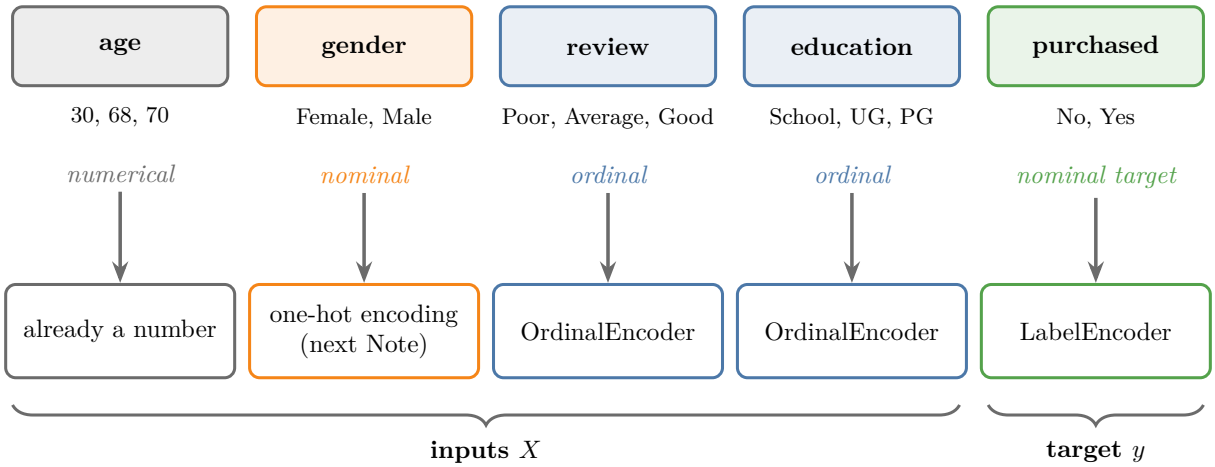


Figure 7: Each column of the customer data and the encoder it needs

Applying a different encoder to different columns by hand is tedious. We would have to separate `gender` and one-hot encode it, separate `review` and `education` and ordinal encode them, and then join everything back together.

scikit-learn has a class for exactly this, the **column transformer** (G-415), taught in the

easy way: ColumnTransformer. Until then, we drop `age` and `gender` and keep only `review`, `education` and `purchased`.

Python: Loading the data and keeping the last three columns.

```
import pandas as pd

df = pd.read_csv("data/customer.csv")
# keep columns 2 onwards: review, education, purchased
df = df.iloc[:, 2:]
```

6.3 Split before encoding

Key point: As with scaling, the train-test split comes before the encoding.

Whenever we do a feature transformation, we split the data first. The encoder learns from the training set only, and is then applied to both the training set and the test set. **Standardization** (G-1874) followed the same rule for scaling.

With 20% of the rows held back for testing, the training set has 40 rows and the test set 10.

Python: Splitting the data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df.iloc[:, 0:2],    # inputs: review, education
    df.iloc[:, -1],     # target: purchased
    test_size=0.2, random_state=0)
# X_train: (40, 2), X_test: (10, 2)
```

`df.iloc[:, 0:2]` takes all rows and the columns at positions 0 and 1. `df.iloc[:, -1]` takes all rows and the last column.

6.4 The categories parameter

Key point: `categories` takes one list per column, each written from the lowest category to the highest.

scikit-learn's **`OrdinalEncoder`** (G-1405) class does **ordinal encoding** (G-1404). When we create it, we pass the parameter **`categories`**: a list that holds one list for each **feature** (G-772) we want to encode.

Here we encode two features, so we pass two lists:

- **review:** the worst is Poor, then Average, and the best is Good. Good must get the biggest number and Poor the smallest.
- **education:** School gets the smallest number and PG the biggest.

Python: Creating the encoder with the order of each column.

```
from sklearn.preprocessing import OrdinalEncoder

oe = OrdinalEncoder(categories=[
    ["Poor", "Average", "Good"],    # review
    ["School", "UG", "PG"],         # education
])
```

If we do not pass `categories`, the encoder chooses the order itself, and Poor might get a bigger number than Good. We do not want that: we want the numbers to follow the real order of the **categories** (G-354).

Extra: Without `categories`, the order is not random: `OrdinalEncoder` sorts the categories alphabetically (scikit-learn API, `OrdinalEncoder`). Sorting gives Average = 0, Good = 1, Poor = 2, and PG = 0, School = 1, UG = 2. The bottom half of Figure 6 shows the result: the codes no longer follow the real order. For **ordinal data** (G-1403), always pass `categories`.

6.5 Fit on the training set, transform both

Key point: `fit` learns the categories; `transform` replaces every category with its number.

As with `StandardScaler`, using the encoder has two steps:

- **fit:** learn the categories of each column from the training set.
- **transform:** replace every category with its position in the list.

Python: Encoding both sets.

```
# learn the categories from the training set only
oe.fit(X_train)
# apply them to both sets
X_train = oe.transform(X_train)
X_test = oe.transform(X_test)
```

Figure 8 runs both steps on our data: `fit` learns the two lists, then `transform` fills in the codes one row at a time, for training rows and test rows alike, with the same lists. The first training rows before and after encoding:

review	education	review (encoded)	education (encoded)
Good	PG	2	2
Poor	School	0	0
Poor	PG	0	2
Average	School	1	0

Good got the biggest number, 2, and Poor the smallest, 0. UG gets 1, since it is in the middle

(the second test row in Figure 8). Every row follows the order we gave.

fit on the 40 training rows learns, in our order:
review: Poor → 0, Average → 1, Good → 2
education: School → 0, UG → 1, PG → 2

set	review	education	review code	education code
train	Good	PG		
train	Poor	School		
train	Poor	PG		
train	Average	School		
train	Good	School		
train	Poor	School		
test	Poor	School		
test	Good	UG		

transform: row 8 replaced by its positions
review: Poor → 0, Average → 1, Good → 2
education: School → 0, UG → 1, PG → 2

set	review	education	review code	education code
train	Good	PG	2	2
train	Poor	School	0	0
train	Poor	PG	0	2
train	Average	School	1	0
train	Good	School	2	0
train	Poor	School	0	0
test	Poor	School	0	0
test	Good	UG	2	1

Figure 8: OrdinalEncoder on the customer data. fit learns the category lists from the 40 training rows; transform then writes each row's positions, for six training rows and two test rows. Darker colours mean higher categories.

After `fit`, the encoder stores the categories it learned, in order, in the attribute **`categories_`** (G-354).

> **Python:** Looking at the learned categories.
> ```
> oe.categories_
> # [array(['Poor', 'Average', 'Good']),
> #  array(['School', 'UG', 'PG'])]
> ```
> Position 0 in each array is the category that became 0, position 1 became 1, and so on.

> **Extra:** `transform` returns a NumPy array of decimal numbers (`2.0`, not `2`), without column names. As in putting the column names back, we can wrap it in `pd.DataFrame(..., columns=...)` to get names back.

> **Extra:** If the test set holds a category that never appeared in the training set, `transform` stops with an error. Passing `handle_unknown="use_encoded_value", unknown_value=-1` to `OrdinalEncoder` makes it write -1 for such categories instead (scikit-learn API, `OrdinalEncoder`).

7. Label encoding in scikit-learn

> **Key point:** `LabelEncoder` is meant only for the **target** (G-1949) y; it numbers the **classes** (G-392) 0 to (number of classes − 1), and we cannot choose the order.

7.1 Only for the target

> **Key point:** The scikit-learn documentation says `LabelEncoder` should encode **target** (G-1949) values, y, not the input X.

A common mistake is to use **label encoding** (G-1030) on features too. Many tutorials also say "label encoding" for any numbering of a feature's categories; in scikit-learn's terms, numbering a feature is ordinal encoding, and `LabelEncoder` is for the target.

The scikit-learn documentation is clear about the rule. The documentation describes `LabelEncoder` as a class to "encode target labels with value between 0 and n_classes-1", and says it "should be used to encode target values, i.e. y, and not the input X" (scikit-learn API, `LabelEncoder`).

So the rule is simple:

- an ordinal **feature** (G-772): `OrdinalEncoder`;
- a categorical target: `LabelEncoder`.

With 2 **classes** (G-392), the codes are 0 and 1; with 3 classes, they are 0, 1 and 2.

7.2 Encoding the purchased column

> **Key point:** `LabelEncoder` takes no order; it gave No = 0 and Yes = 1.

`LabelEncoder` (G-1033) takes no parameters. We cannot tell it the order: it decides by itself which class gets which number. The steps are the same as before: fit on the training **target** (G-1949), then transform both targets.

> **Python: Label encoding** (G-1030) the target.
> ```
> from sklearn.preprocessing import LabelEncoder
>
> le = LabelEncoder()
> # learn the classes from the training target
> le.fit(y_train)
> le.classes_
> # array(['No', 'Yes'])
>
> y_train = le.transform(y_train)
> y_test = le.transform(y_test)
> ```
> The attribute **`classes_`** (G-392) lists the classes in order: position 0 is No, so No becomes 0 and Yes becomes 1.

The first five training targets were Yes, Yes, No, No, No. After encoding they are 1, 1, 0, 0, 0.

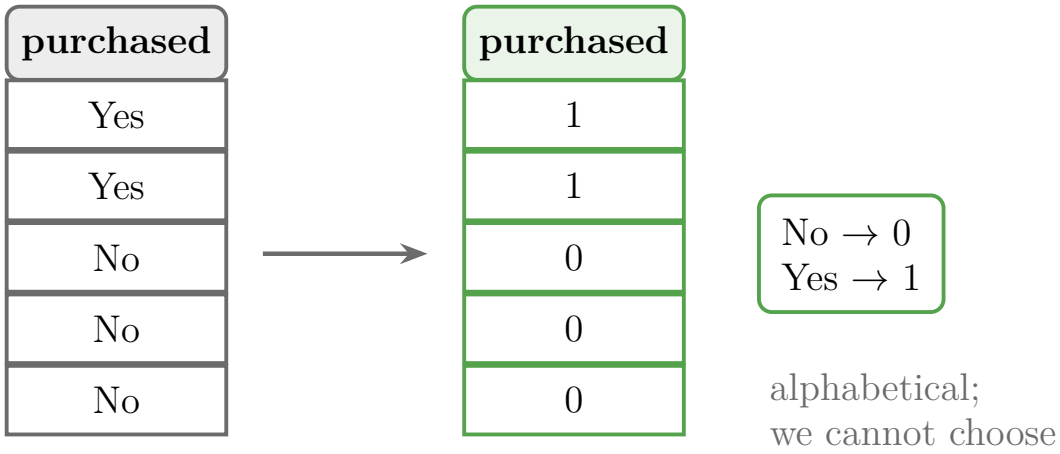


Figure 9: `LabelEncoder` on the first five training targets

Figure 9 shows these five targets before and after.

> **Extra:** `LabelEncoder` also sorts the classes alphabetically, which is why No comes before Yes. For a target the order does not matter: a classifier only needs a different number for each class. `le.inverse_transform([0, 1])` turns the numbers back into `['No', 'Yes']`, which is useful for reading a model's predictions.

8. Summary

	Ordinal encoding	Label encoding	One-hot encoding
Used on	ordinal features (X)	the categorical target (y)	nominal features (X)
Example column	review, education	purchased	gender, state
Result	one column of 0, 1, 2, ...	one column of 0, 1, 2, ...	one new column per category
Order of codes	we give it with `categories`	chosen by the encoder (alphabetical)	no order
scikit-learn class	`OrdinalEncoder`	`LabelEncoder`	next Note
Learned categories in	`categories_`	`classes_`	

- Data is numerical or categorical; categorical data is nominal (no order) or ordinal (has an order). The kind decides the encoder, because numbers carry an order: on nominal **categories** (G-354) such as states they invent an order that a model then uses to group them.
- ML algorithms need numbers, so categories stored as text must be encoded before training.
- **Ordinal encoding** (G-1404) replaces ordered categories by 0, 1, 2, ..., following the order we give it, so the codes keep the real ranking (School < UG < PG) that the model can use.
- Always pass `categories` to `OrdinalEncoder`, because the encoder cannot know what "UG" or "PG" mean; otherwise the order is alphabetical and the real order is lost.
- **Label encoding** (G-1030) does the same job, but only for the target y; never use it on the **features** (G-772), because it takes no order (the scikit-learn documentation says it is for y, not X). For a **target** (G-1949) the order does not matter: a classifier only needs a different number for each class.
- Split first; fit the encoder on the training set; transform both sets, so the encoder learns only from the training data, the same rule as for scaling.
- So to turn categories into numbers: ordered features get ordinal encoding, a categorical target gets label encoding, and nominal features get one-hot encoding.

9. Sources

Built from

- CampusX, "Encoding Categorical Data | Ordinal Encoding | Label Encoding", YouTube, https://www.youtube.com/watch?v=w2GglmYHfmM
- StatQuest with Josh Starmer, "One-Hot, Label, Target and K-Fold Target Encoding, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=589nCGeWG1w (the grouping example of section 3.1)

Other references

- scikit-learn API reference. `sklearn.preprocessing.OrdinalEncoder` and `sklearn.preprocessing.LabelEncoder`. scikit-learn.org.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Label encoding (G-1030)	Replacing the classes of the target by 0, 1, 2, ...; for the output column only.
categories_ (G-354)	The attribute holding the categories `OrdinalEncoder` learned, in order.
Ordinal encoding (G-1404)	Replacing ordered categories by 0, 1, 2, ... in their order; for input columns.
Nominal data (G-1330)	Categorical data whose categories have no order, such as states.
Ordinal data (G-1403)	Categorical data whose categories have a natural order, such as Poor < Average < Good.
OrdinalEncoder (G-1405)	scikit-learn's class for ordinal encoding; takes the order through `categories`.
classes_ (G-392)	The attribute holding the classes `LabelEncoder` learned, in order.
LabelEncoder (G-1033)	scikit-learn's class for label encoding the target.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Standardization (G-1874)	Scaling a column to mean 0 and standard deviation 1 (subtract the mean, divide by the standard deviation), so features measured on different scales become comparable.
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.
Numerical data (G-1367)	Data made of numbers.
Categorical data (categorical column) (G-351)	Data made of categories: labels rather than numbers.

Term	Meaning
Decision tree (G-561)	A model that predicts by asking a chain of questions about the input columns: nested if-else conditions.
Classification (G-395)	Predicting which of a fixed set of categories (classes) a row belongs to, learned from examples whose answers are known (supervised learning with a categorical output).
Column transformer (G-415)	A scikit-learn class that applies different transformations to different columns at once and joins the results.
Encoding (G-685)	Two senses: the rulebook that maps text characters to stored bytes (Video 15); or turning categories into numbers (categorical encoding, Video 26).